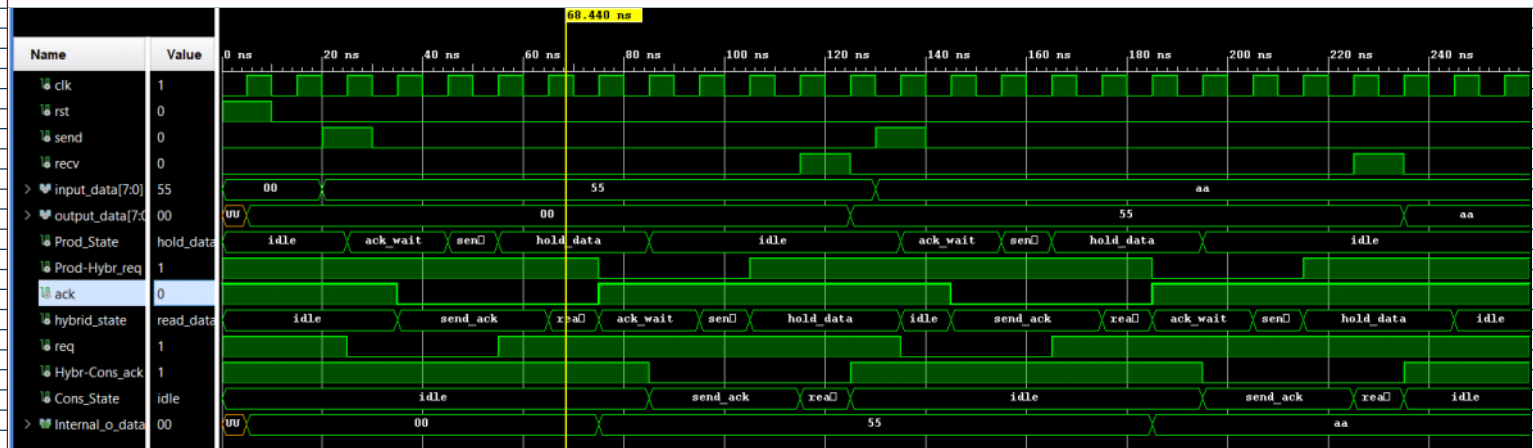
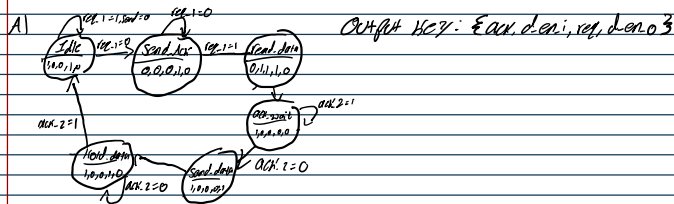


Homework #3

Friday, September 25, 2026 9:48 PM



1. Slice Logic

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	9	0	63400	0.01
LUT as Logic	9	0	63400	0.01
LUT as Memory	0	0	19000	0.00
Slice Registers	32	0	126800	0.03
Register as Flip Flop	32	0	126800	0.03
Register as Latch	0	0	126800	0.00
F7 Muxes	0	0	31700	0.00
F8 Muxes	0	0	15850	0.00

21 of the FF come from the producer and consumer blocks. The producer has 8 data register FF and 2 state register FF. Then the consumer has 3 state register FF and 8 data register FF. That leaves 11 FF unaccounted for. The Hybrid block has 11 FF, split into 8 data register FF and 3 state FFs

The data registers for the Hybrid are the internal register that was asked for. It has 3 state register FFs because it has 6 states which needs at least 3 bits to render in binary ($11 = 3, 110 = 6$). The same can be said for the producer FF since it has 4 states which only need 2 bits ($11, 10, 01, 00$). The consumer however uses onehot which means each state is encoded by a FF, since cons has 3 states it needs 3 FFs.

Hybrid:

-- One process for state transitions, two processes for outputs, + one data path process

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
```

entity TwoWayHandshake is

Port (

i_clk : in std_logic; -- clock
i_rst : in std_logic; -- reset

i_data : in std_logic_vector(7 downto 0); -- data

i_req : in std_logic; -- request
o_ack : out std_logic; -- acknowledge

o_req : out std_logic; -- request
i_ack : in std_logic; -- acknowledge

o_data : out std_logic_vector(7 downto 0) -- data out

);

end TwoWayHandshake;

architecture Behavioral of TwoWayHandshake is

type state_type is (idle, send_ack, read_data, ack_wait, send_data, hold_data);
signal hybrid_state : state_type;

signal s_d_en_i : std_logic; -- for the input data latch
signal s_d_en_o : std_logic; -- for the output data latch

```

begin
-- State machine transitions process
process (i_clk)
begin
if rising_edge(i_clk) then
if (i_rst = '1') then
hybrid_state <= idle;
else
case hybrid_state is
when idle =>
if i_req = '0' then
hybrid_state <= send_ack;
else
hybrid_state <= idle;
end if;
when send_ack =>
if i_req = '1' then
hybrid_state <= read_data;
else
hybrid_state <= send_ack;
end if;
when read_data =>
hybrid_state <= ack_wait;
when ack_wait =>
if i_ack = '0' then
hybrid_state <= send_data;
else
hybrid_state <= ack_wait;
end if;
when send_data =>
hybrid_state <= hold_data;
when hold_data =>
if i_ack = '1' then
hybrid_state <= idle;
else
hybrid_state <= hold_data;
end if;
end case;
end if;
end if;
end process;

-- State machine outputs
process (hybrid_state)
begin
case hybrid_state is
when ack_wait | send_data => o_req <= '0';
when others => o_req <= '1';
end case;
end process;

process (hybrid_state)
begin
case hybrid_state is
when send_data => s_d_en_o <= '1';
when others => s_d_en_o <= '0';
end case;
end process;

process (hybrid_state)
begin
case hybrid_state is
when read_data => s_d_en_i <= '1';
when others => s_d_en_i <= '0';
end case;
end process;

process (hybrid_state)
begin
case hybrid_state is
when send_ack | read_data => o_ack <= '0';
when others => o_ack <= '1';
end case;
end process;
-----

-- Data path process
process (i_clk)
begin
if rising_edge(i_clk) then
if (i_rst = '1') then
o_data <= (others=>'0');
else
if s_d_en_o = '1' then
o_data <= i_data;
end if;
if s_d_en_i = '1' then
o_data <= i_data;
end if;
end if;
end if;
end process;

end Behavioral;

```

```

Top_Level:

-- Wrapper entity for producer and consumer

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity handshakeTop is
Port (
    i_clk : in std_logic;        -- clock
    i_rst : in std_logic;        -- reset

    i_send : in std_logic;        -- initiate send
    i_data : in std_logic_vector(7 downto 0); -- data in

    o_rcv : out std_logic;        -- received something
    o_data : out std_logic_vector(7 downto 0) -- data out
);
end handshakeTop;

architecture Behavioral of handshakeTop is
    component producer is
    port (
        i_clk : in std_logic;        -- clock
        i_rst : in std_logic;        -- reset

        i_send : in std_logic;        -- initiate send
        i_data : in std_logic_vector(7 downto 0); -- data in

        o_req : out std_logic;        -- request
        i_ack : in std_logic;        -- acknowledge
        o_data : out std_logic_vector(7 downto 0) -- data out
    );
    end component;

    component consumer is
    port (
        i_clk : in std_logic;        -- clock
        i_rst : in std_logic;        -- reset

        i_req : in std_logic;        -- request
        o_ack : out std_logic;        -- acknowledge
        i_data : in std_logic_vector(7 downto 0); -- data out

        o_rcv : out std_logic;        -- recieved something
        o_data : out std_logic_vector(7 downto 0) -- data out
    );
    end component;

    component TwoWayHandshake is
    port (
        i_clk : in std_logic;        -- clock
        i_rst : in std_logic;        -- reset

        i_data : in std_logic_vector(7 downto 0); -- data

        i_req : in std_logic;        -- request
        o_ack : out std_logic;        -- acknowledge

        o_req : out std_logic;        -- request
        i_ack : in std_logic;        -- acknowledge

        o_data : out std_logic_vector(7 downto 0) -- data out
    );
    end component;

    signal req : std_logic;
    signal ack : std_logic;
    signal ack_i : std_logic;
    signal req_o : std_logic;
    signal internal_data : std_logic_vector(7 downto 0);
    signal hybrid_internal_data : std_logic_vector(7 downto 0);

begin
    prod : producer port map (
        i_clk => i_clk,
        i_rst => i_rst,
        i_send => i_send,
        i_data => i_data,
        o_req => req,
        i_ack => ack,
        o_data => internal_data
    );

    cons : consumer port map (
        i_clk => i_clk,
        i_rst => i_rst,
        i_req => req_o,
        o_ack => ack_i,
        i_data => hybrid_internal_data,
        o_rcv => o_rcv,
        o_data => o_data
    );

```

```

hybrid: TwoWayHandshake port map (
    i_clk => i_clk,
    i_rst => i_rst,
    i_req => req,
    o_ack => ack,
    o_req => req_o,
    i_ack => ack_i,
    i_data => internal_data,
    o_data => hybrid_internal_data
);
end Behavioral;

TB:

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity handshake_tb is
-- Port ();
end handshake_tb;

architecture Behavioral of handshake_tb is
    component handshakeTop is
        port (
            i_clk : in std_logic;          -- clock
            i_rst : in std_logic;          -- reset

            i_send : in std_logic;          -- initiate send
            i_data : in std_logic_vector(7 downto 0); -- data in

            o_rcv : out std_logic;          -- received something
            o_data : out std_logic_vector(7 downto 0); -- data out
        );
    end component;

    signal clk : std_logic := '0';
    signal rst : std_logic := '0';

    signal send : std_logic := '0';
    signal rcv : std_logic;
    signal input_data : std_logic_vector(7 downto 0) := (others => '0');
    signal output_data : std_logic_vector(7 downto 0);

    constant clk_period : time := 10 ns;
begin
    uut : handshakeTop port map (
        i_clk => clk,
        i_rst => rst,
        i_send => send,
        i_data => input_data,
        o_rcv => rcv,
        o_data => output_data
    );

    clk <= not clk after clk_period/2;

    stim_proc : process
    begin
        rst <= '1';
        wait for clk_period;
        rst <= '0';
        wait for clk_period;

        input_data <= x"55";
        send <= '1';
        wait for clk_period;
        send <= '0';
        wait for clk_period*10;

        input_data <= x"AA";
        send <= '1';
        wait for clk_period;
        send <= '0';
        wait for clk_period*12;

        report "Simulation Finished" severity FAILURE;
    end process;
end Behavioral;

```